

# COMP20005 Workshop Week 9

1

Representation of integers and Representation of floats

2

2D arrays

3

Discuss: Array Search -- Finding a specific value from an array (Ex 7.7, 7.8)

**LAB**

- Array exercises: Consolidate your understanding of arrays, by completing Exercises 7.7, 7.8, and 7.15. *Arrays, and being able to manipulate them, is a key subject.*

Assignment 2:

- released this week, due Fri week 11
- be ready for discussion in workshop week 10

# Part 1: A number can be written using different bases

- Base **10** (decimal system) uses 10 digits 0..9      10 is written using 2 digits: **10**
- Base **2** (binary)      uses 2 digits 0..1      2 is written using 2 digits: **10**
- Base **16** (hexadecimal)      uses 16 digits 0..9, A..F      16 is written using 2 digits: **10**

*Converting from any base B (eg. B=2, 16) to decimal is simple:*

the number      ...  $d_3$   $d_2$   $d_1$   $d_0$  .  $d_{-1}$   $d_{-2}$  ... <sub>(B)</sub>      (in base **B**)

has the value      ... +  $d_3 \times B^3$  +  $d_2 \times B^2$  +  $d_1 \times B^1$  +  $d_0$  +  $d_{-1} \times B^{-1}$  +  $d_{-2} \times B^{-2}$  ...

Examples:

$$1011.101_{(2)} = ?_{(10)}$$

$$3A.2_{(16)} = ?_{(10)}$$

# Converting Numbers from Decimal → Binary → Hexadecimal

*Excercise: Write the following numbers in the binary and hexadecimal formats:*

| Decimal format | Binary Format | Hexadecimal format |
|----------------|---------------|--------------------|
| 9              |               |                    |
| 59             |               |                    |
| 197            |               |                    |
| 5.875          |               |                    |

Notes:

- better to convert the integer and the fractional parts separately
- **binary  $\leftrightarrow$  hexadecimal**: one hexadecimal digit is equivalent to 4 binary digits

|       |       |       |       |       |       |       |       |          |          |          |
|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|
| $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ | $2^{-1}$ | $2^{-2}$ | $2^{-3}$ |
| 128   | 64    | 32    | 16    | 8     | 4     | 2     | 1     | 0.5      | 0.25     | 0.125    |

# Two-compliment representation of integers using $w$ bits

- Note that we use a fixed amount of bits  $w$
- $w$  bits gives  $2^w$  combinations  $\rightarrow$  possible range :  $-2^{w-1} .. +(2^{w-1}-1)$ 
  - -128 to +127 if  $w=8$
  - $-2^{31}$  to  $+2^{31}-1$  if  $w=32$

Representation of a signed integers :

- Positive number  $x$ : as the binary representation of  $x$  in  $w$  bits. Note: the first bit is always 0
- Negative numbers  $x$ : as the *twos-complement* of (the representation) of  $|x|$  in  $w$  bits. The first bit will always be 1 and has the value  $-2^{w-1}$

# A trick: Finding twos-complement representation in w bits

Suppose that we need to find the twos-complement representation of a *negative*  $x$  in  $w=16$  bits. Do it in 3 steps:

- 1) Write binary representation of  $|x|$  in  $w$  bits
- 2) Find the **rightmost one-bit**
- 3) Inverse (ie. flip 1 to 0, 0 to 1) all bits **on the left** of that **rightmost one-bit**

| <i>find the 2-comp repr of -36</i>               | Bit sequence |             |             |               |
|--------------------------------------------------|--------------|-------------|-------------|---------------|
| 1) bin repr of 36 in 16 bits                     | 0000         | 0000        | 0010        | 0100          |
| 2) find the <b>rightmost 1</b> -bit              | 0000         | 0000        | 0010        | 0 <b>1</b> 00 |
| 3) <b>inverse</b> the left of that <b>1</b> -bit | <b>1111</b>  | <b>1111</b> | <b>1101</b> | <b>11</b> 00  |

→ -36 is represented as the bit pattern 1111 1111 1101 1100,  
or FFDC in the hexadecimal format

*Understand why by adding the bit patterns for 36 and -36*

## Exercise 1: 2-complement representation in w=16 bits

Q: What are 17, -17, 34, and -34 as 16-bit twos-complement binary numbers, when written as (a) binary digits, and (b) hexadecimal digits?

| Decimal | twos-complement as binary digits | twos-complement as hexadecimal digits |
|---------|----------------------------------|---------------------------------------|
| 17      |                                  |                                       |
| -17     |                                  |                                       |
| 34      |                                  |                                       |
| -34     |                                  |                                       |

# Representation of floats

There are several methods. Examples include:

- the method described in [numericA.pdf](#) and in the text book (using 16 bits in total)
- another is an [IEEE standard](#), which is:
  - employed in most of modern computers,
  - demonstrated in the lecture, and
  - you can find/experiment with using the program [floatbits.c](#) ([numericA.pdf](#) p.22).

# Representation of float x: the principle

Convert  $|x|$  to binary form, and transform so that

- Option a:  $|x| = 0.\mathbf{b_0}b_1b_2... \times 2^e$  where  $b_0 = 1$
- Option b:  $|x| = \mathbf{1}.b_1b_2... \times 2^e$  where the integer part is 1,  $b_0 = 1$

so  $|x|$  can be represented as 2 integers  $(e, m)$  where:

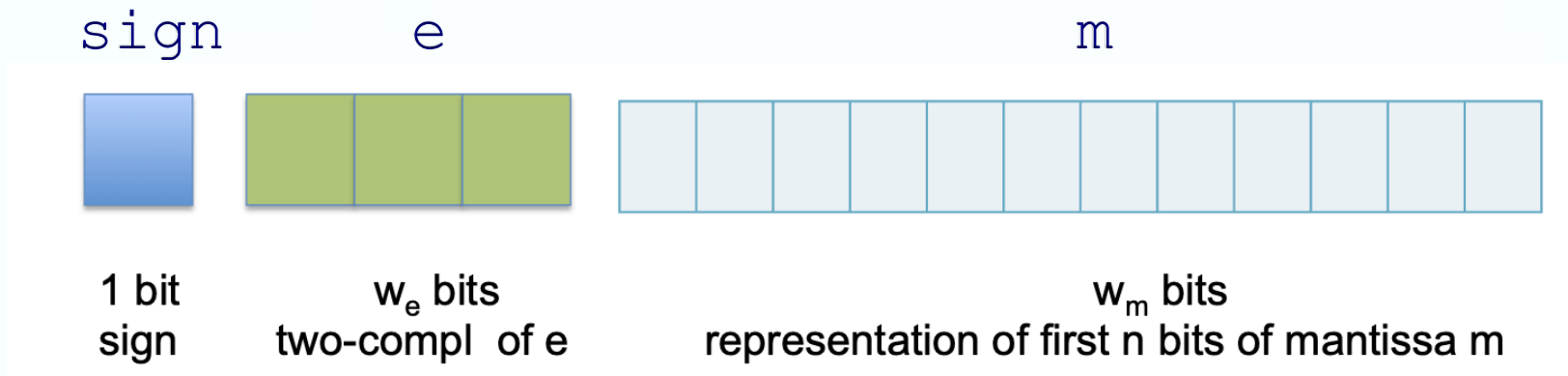
- for Option a:  $m = b_0b_1b_2...$
- for Option b:  $m = b_1b_2...$  (which hidden  $b_0$  since  $b_0$  is always 1)

Then,  $x$  is represented as triple  $(sign, \text{exponent } e, \text{mantissa } m)$ , and:

- using 1 bit for  $sign$  (0: positive, 1: negative)
- using  $w_e$  bit for exponent (note  $e$  can be negative)
- using  $w_m$  bits for mantissa (with possible cutting off right bits or adding 0-bits to the right of mantissa to fit into  $w_m$  bits)



## Example 1: Representation of floats as described in numericA.pdf



Convert  $|x|$  to binary form, and transform (using Option A) so that:

$$|x| = 0.\mathbf{b_0}b_1b_2... \times 2^e \quad \text{where } b_0 = 1$$

$$m = b_0b_1b_2...$$

**Note:** just an example, not important, no need to remember the above details!

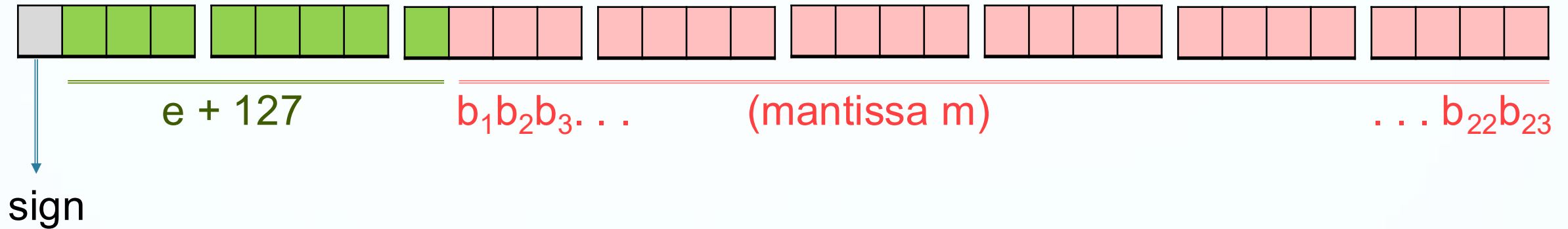
## Exercise 2

**Class Exercise:** using the described 12-bit representation (page 21 of the numericA.pdf slides), what are the floating point representations of -3.125, 17.5, and 0.09375, when written as (a) binary digits, and (b) hexadecimal digits?

Do it, then check your answer with your classmates or tutor:

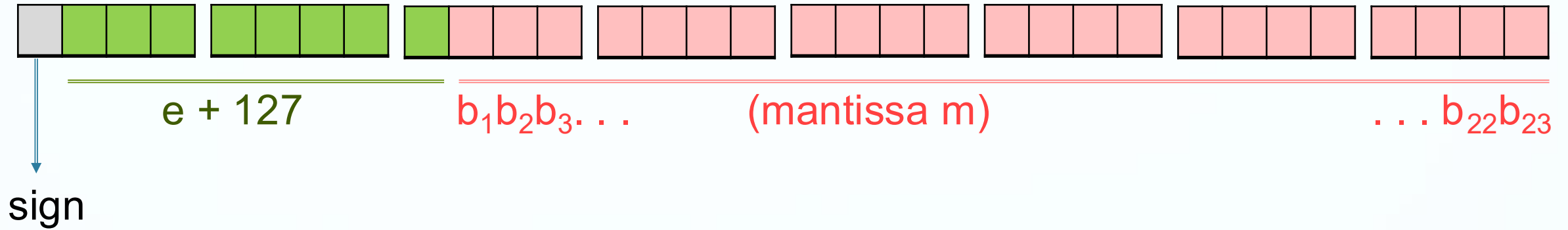
| value   | Binary | Hexadecimal |
|---------|--------|-------------|
| -3.125  |        |             |
| 17.5    |        |             |
| 0.09375 |        |             |

## Example 2: Representation of 32-bit float: (IEEE 754, as in floatbits.c )



- transform  $|x|$  (using Option B) to  $|x| = \mathbf{1}.b_1b_2\dots \times 2^e$  where the integer part  $b_0 = 1$
- represent  $x$  as a triple (*sign*, exponent  $e$ , mantissa  $m = b_1b_2\dots$ )
- using  $w_e = 8$  bit for  $e$ :
  - $e$  is represented as the (unsigned) bit pattern for the value  $127+e$
  - so:
    - $e = 5$  is represented as ???
    - $e = -2$  is represented as ???
- using  $w_m = 23$  bits for  $m$

# Representation of 32-bit float: (IEEE 754, as in floatbits.c )



## Notes:

- Remember that  $w_e = 8$
- The exponent  $e$  is represented using an 8-bit pattern encoding the positive integer  $e + 127$

## Un-important Notes, just for fun:

- we say that  $e$  is represented in **excess-127** format
- $e$  can be  $-126 \rightarrow +126$ , corresponding to the bit pattern **0000 0001**  $\rightarrow$  **1111 1110**.
- zero is represented as 32 **zero-bit**, with the special value **0000 0000** for  $e$
- infinity is represented as 32 **one-bit**, with the special value **1111 1111** for  $e$

# Representation of 32-bit float: (IEEE 754, as in floatbits.c )

$$w_s=1, w_e=8, w_m=23 \quad |x| = 1.b_1b_2... \times 2^e$$

Example:  $x=6.5$

In binary:  $x=110.1 = 1.101 \times 2^2$

→ sign bit: 0

→  $e=2 \Rightarrow 127+2=128+1$  is represented as 1000 0000 + 1

→  $e$  is represented as 1000 0001

→ mantissa: 101 00...0

→ Final representation:

0100 0000 1101 0000 0000 0000 0000 0000

or 4 0 D 0 0 0 0 0 (16)

## Exercise 2 (modified)

**Group Exercise:** using 32-bit representation in modern PC (an IEEE standard), what are the floating point representations of -3.125, 17.5, and 0.09375, when written as (a) binary digits, and (b) hexadecimal digits?

Do it, then check your answer with your classmates or tutor:

| value   | Binary | Hexadecimal |
|---------|--------|-------------|
| -3.125  |        |             |
| 17.5    |        |             |
| 0.09375 |        |             |

- transform binary number to  $1.??? \times 2^e$
- $w=32$ ,  $w_e=8$ , the bit pattern for  $e=1$  is 1000 0000

## Part 2: 2D arrays

Each data type has a corresponding pointer type, for example

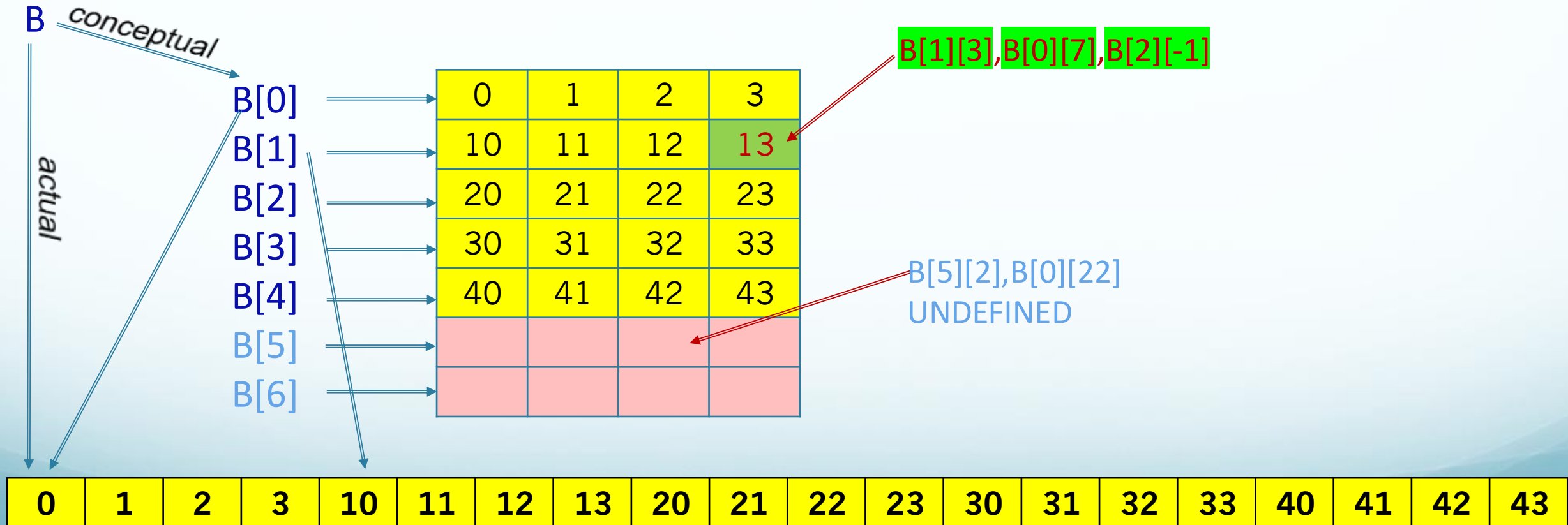
```
int A[5], B[3][4];
```

```
// B is an array of "array of 4 int"
```

if A is an 1D array, i is an int, A[i] is valid, but might be undefined!  
if B is an 2D array, B[i][j] is valid, but might be undefined!

```
int B[5][4]={0,1,2,3},{10,11,12,13},{20,21,22,23},{30,31,32,33},{40,41,42,43};
```

- B is an array of arrays, each B[i] is an array (and a pointer)
- B[i] points to B[i][0] , ie. B[i] has the value of &B[i][0]





# example

```
int A[5]={-7,3,5,2,1};
```

```
int B[3][5]
```

```
={{1,12,3,14,5},{15,4,6,2,13},{10,7,8,11,9}};
```

What's the value of

1.  $B[A[4]][A[1]]$
2.  $B[A[1]][A[4]]$
3.  $B[A[4]][A[2]]$
4.  $B[A[3]][A[2]]$   $B[2][5]$
5.  $B[A[3]][A[0]]$   $B[2][-7]$

## Part 3: Searching in Arrays

**General task:** given an array of  $n$  elements  $A[n]$ , find a specific element.

*Example:*

1. Find a specific value that satisfies a condition, such as:
  - a) being equal to  $x$ , ie. find  $i$  such that  $A[i] == x$
  - b) find the value of  $k$ -smallest, or  $k$ -largest element
  
2. Find some “extreme” value, such as:
  - a) find the maximal/minimal element
  - b) find the most frequent element

# Searching for a specific element

Task: search for a particular value in an array

## Examples

- Return index  $i$  such that  $A[i] == x$  (or return -1 if NOTFOUND): *there is a chance for NOTFOUND*
- Exercise 7.08 (k-smallest)

```
#define NOTFOUND -1

// return the first index i that A[i]==x
int search(int A[], int n) {
    ???
    for (i=0; i<n; i++) {
        // process A[i]: test if A[i] is the solution
        //           return i or A[i] if YES
    }
    return NOTFOUND;
}
```

# Searching for an “extreme” element

## Examples

- search for the maximal (or minimal) value in an array.
- exercise 7.07 (most-frequent)

Note: *solution always found, ie. there is normally no NOTFOUND case*

```
// return a kind of min/max value in A
// pay attention when there are multiple soln
```

```
int find(int A[], int i) {
    // soln= first estimation
    for (i=0; i<n; i++) {
        // process A[i]:
        // test if A[i] is a better solution
        // if YES, update soln
    }
    return soln;
}
```

- Discuss: how to approach 7.07, 7.08 (finding a value in an array)
- (time permits): 7.03 or 7.15
- Manipulating arrays, including:
  - Array exercises: completing Exercises 7.7, 7.8, 7.03, and 7.15.
    - 7.07: find most frequent value
    - 7.08: find k-smallest
    - 7.03: sort & remove duplicates
    - 7.15: anagrams
  - Other exercises in Chapter 7
  - Remember: *Arrays, and being able to manipulate data stored in them, are the absolute backbone of this subject.*
- [If you want a challenge] Write a program that outputs the bit-patterns associated with 64-bit double values.

# Additional Material

- Procedures for converting between from decimal to binary representation

# Procedure for Decimal $\rightarrow$ Binary: Integer Part

*Changing integer  $x$  to binary (when the solution not obvious):*

*Just divide  $x$  and the subsequent quotients by 2 until getting zero. The sequence of remainders, in **reverse order** of appearance, is the binary form of  $x$ . 6567*

*Example: 23*

| operation | quotient | remainder |
|-----------|----------|-----------|
| 23 : 2    | 11       | 1         |
| 11 : 2    | 5        | 1         |
| 5 : 2     | 2        | 1         |
| 2 : 2     | 1        | 0         |
| 1 : 2     | 0        | 1         |

So:  $23 = 10111_{(2)}$   $11 =_{(2)}$   $46 =_{(2)}$

# Procedure for Decimal $\rightarrow$ Binary: Fraction Part

*Fraction: Multiply it, and subsequent fractions, by 2 until getting zero. Result= sequence of integer parts of results, in appearance order. Examples:  $0.1+01+\dots+0.1$*

| 0.375           |     |          | 0.1           |     |          |
|-----------------|-----|----------|---------------|-----|----------|
| operation       | int | fraction | operation     | int | fraction |
| $.375 \times 2$ | 0   | .75      | $.1 \times 2$ | 0   | .2       |
| $.75 \times 2$  | 1   | .5       | $.2 \times 2$ | 0   | .4       |
| $.5 \times 2$   | 1   | .0       | $.4 \times 2$ | 0   | .8       |
|                 |     |          | $.8 \times 2$ | 1   | .6       |
|                 |     |          | $.6 \times 2$ | 1   | .2       |

So:  $0.375 = 0.011_{(2)}$

$0.1 = 0.00011(0011)_{(2)}$